

Agile Software Process and Its Experience

Mikio Aoyama

Department of Information and Electronics Engineering
Niigata Institute of Technology
1719 Fujihashi, Kashiwazaki 945-11, Japan
Tel: +81-257-22-8129, E-mail: mikio@iee.niit.ac.jp

ABSTRACT

This article proposes a new software process model, *ASP* (*Agile Software Process*) and discusses its experience in a large-scale software development.

The Japanese software factory was a successful model in the development of quality software for large-scale business applications in the 80s. However, the requirements to software development have been dramatically changed for last several years. Development cycle-time has been promoted to one of the top goals of software development in the 90s.

Unlike conventional software process models based on volume, the ASP is a time-based process model which aims at quick delivery of software products by integrating the light-weight processes, modular process structures and incremental and iterative process enaction. The major contributions of APS include;

- 1) A new process model and its enaction mechanism based on time,
- 2) A software process model for evolutionary delivery,
- 3) A software process architecture integrating concurrent and asynchronous processes, incremental and iterative process enaction, distributed multi-site processes, and the people-centered processes,
- 4) A process-centered software engineering environment for ASP, and
- 5) Experience and lessons learned from the use of ASP in the development of a family of large-scale communication software systems for more than five years.

Keywords

Software Process, Project Management, Evolution, Process centered Environment, and Distributed Development

1 INTRODUCTION

The notion of *software process* has long been one of the major topics in software engineering. After Royce's *waterfall model*, various enhancements on the model were proposed. For example, *iterative enhancement* captured the

evolution of large-scale software development [Basi75]. In the mid 80s, several new process models were proposed [Agre86]. From the risk management point of view, Boehm proposed the *spiral model* [Boeh86]. Then, the concept of *process programming* opened the door to the research on formal modeling techniques of software process [Oste87]. Software process continues to be an important topic in the software engineering community [Fink94, Hump89].

The notion of software process has been appreciated in Japan too. It played diverse roles at the heart of *Japanese software factories* [Cusu91]. However, the software process is not explicit but embedded in various techniques such as *CPI* (*Continuous Process Improvement*) and *TQC* (*Total Quality Control*) [Aoya96a, Naka92].

At Fujitsu, the author has been involved in the development of a family of large-scale telecommunication software systems for last ten years. In this project, we experienced an evolutionary change of our software processes [Aoya87, Aoya95, Aoya96a]. Since the development cycle-time was promoted to one of the most important goals of the project, we have entirely changed our process model from a *sequential centralized process* to a *distributed concurrent process* as illustrated in Fig. 1

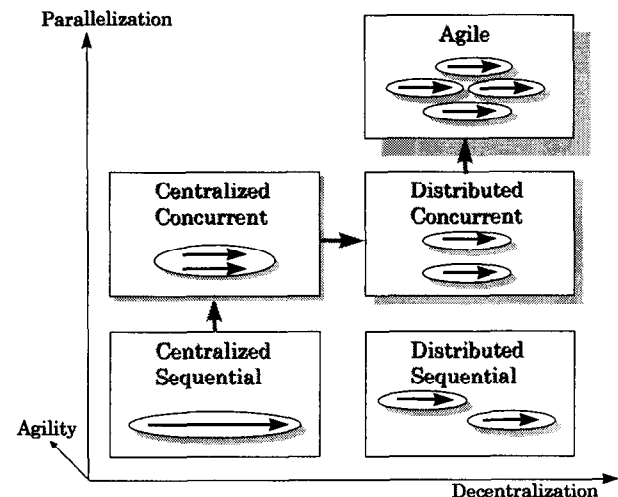


Fig. 1 Evolution of Software Process

With a distributed concurrent process model, multiple functions for a family of software systems are concurrently developed by multiple development teams at the multiple development sites geographically distributed. The two dimensional changes on software process have required us

to fundamentally re-think every aspect of software development activities ranging from individual developers to entire project [Aoya93].

As time-to-market has been promoted to one of the most important goals in product development, the concept of *agility* has been extensively discussed in hardware production [Gold95]. However, little discussion has been given to software development.

This article proposes the *ASP (Agile Software Process)* model based on our experiences in the distributed concurrent development, lessons learned in Japanese software factories and various concepts in hardware production processes.

2 THE AGILE SOFTWARE PROCESS

The ASP does not simply mean rapid application development. Rather, it emphasizes rapid and flexible adaptation to changes of the process, product and environment. Therefore, ASP is not a single technique but a total technology based on a coherent concept. At the heart of the ASP system, the concept of the ASP model is rather different from conventional process models. We characterize the ASP model as follows.

(1) Incremental and Evolutionary Process

Agility means quick adaptations to changes in requirements and surrounding environments. Conventional processes are based on a *Bigbang-deliver and maintenance model* and consist of a monolithic and fat process, which is hard to change. Opposed to this, the ASP is based on an *incremental-delivery and evolution model* by which products are incrementally delivered over time. Thus, the ASP model consists of a number of *light-weight* processes which are small and manageable units.

(2) Modular and Lean Process

The ASP model, consisting of light-weight processes, is more modular and leaner than conventional process models.

(3) Time-Based Process

The structure and enaction of ASP is based on time, which is different from conventional volume-based process. The enaction of ASP is iterative with fixed cycle-time. Therefore, large-volume development can be divided into multiple releases so that they can be developed incrementally and concurrently in a predictive way.

3 ECONOMIC FOUNDATIONS OF ASP

Before going into the details, we discuss the mathematical foundation of process economics of ASP.

To modularize the software process, we have introduced two new concepts of process metrics: *time economics* and *volume economics* of software process.

While time economics define the efficiency of the software process in terms of time, volume economics define it in terms of the size of products.

3.1 Time Economics: Optimal Timing for Concurrent and Cyclic Processes

We have found the optimal economic process model of ASP. As illustrated in Fig. 2, we assume that the unit process model consists of two major parts: upper stream and lower stream. α denotes the ratio of upper stream. Then, conventional waterfall model can be divided into N unit-processes, that is $T_{CONV} = N \times T_{UNIT}$. The unit-processes are enacted incrementally and concurrently. β denotes the concurrency ratio of the ASP between the successive release. Note that $\beta \leq \alpha$.

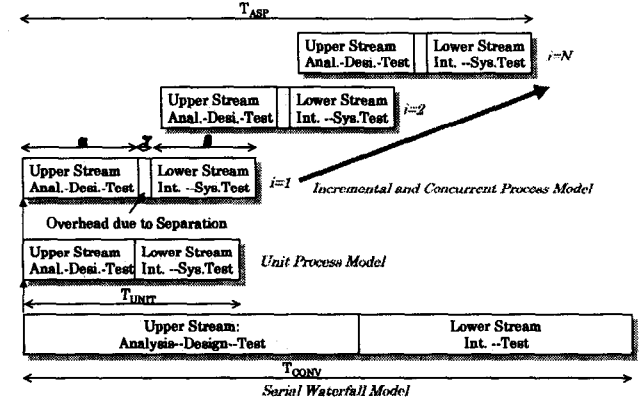


Fig. 2 Process Economic Model

We assume that incremental and concurrent enaction requires some extra effort to integrate the releases due to the coupling among the incremental and concurrent processes of the ASP. γ denotes such overhead.

Let T_{ASP} denote development time for ASP. We can calculate the reduction ratio of development-time R as follows.

$$R = T_{ASP} / T_{CONV} = ((1 + \gamma) T_{UNIT} + \sum_{i=2}^N (1 - \beta + \gamma) T_{UNIT}) / T_{CONV} \\ = 1 - \beta (N - 1) / N + \gamma \quad (1)$$

It can be shown that $\alpha = 0.5$ and $\beta = 0.5$ minimize the development-time and the minimum cycle-time is 0.5 if $\gamma = 0$. Fig. 3 depicts the numerical calculation of equation (1) for the case of $\beta = 0.5$ and $\beta = 0.25$ while $\alpha = 0.5$.

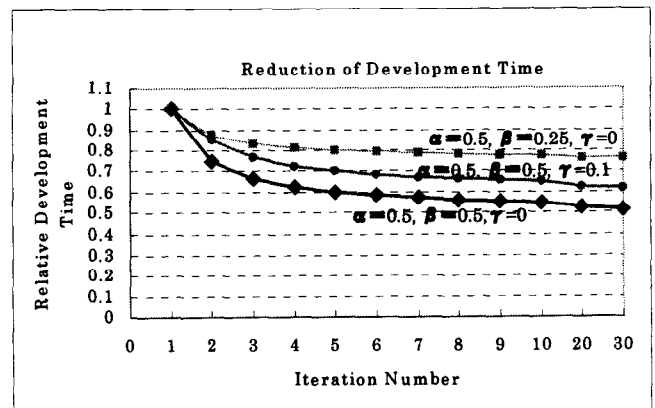


Fig. 3 Reduction of Cycle-Time

However, due to the coupling between releases, the cycle time is extended. Pittman empirically estimated such coupling in terms of cost [Pitt93]. He suggested 7% extra cost combining both incremental and iterative processes. Here, we used more conservative extra cost of 10%, that is $\gamma=0.1$.

From this cost model, we can deduce the following three propositions.

Proposition 1

The development-time is minimum when the upper stream and lower stream is divided at the middle of the unit life-cycle, that is $\alpha=0.5$.

Proposition 2

The development-time is minimum when the concurrency ratio β is 0.5, that is each release is pipelined at the middle of the unit life-cycle.

Proposition 3

Due to the coupling cost, the development-time is gradually extended with the increase of number of releases.

3.2 Volume Economics: Optimal Size of Unit Process

It is well known that increasing the size of software to be developed decreases the productivity. We call this characteristic volume economics. To implement the ASP, we modularize software processes and products so that large-scale software can be developed as a collection of small units with a narrow range of productivity variation. Therefore, we need to identify an optimal unit size of development.

As discussed in the time economics model, dividing the software into small units requires extra efforts. Therefore, we can assume the productivity loss proportional to the number of separations, that is, $L = \delta \times N$, where δ is an appropriate coefficient. Taking into consideration such loss, we can deduce an adjusted COCOMO model [Boeh81] for

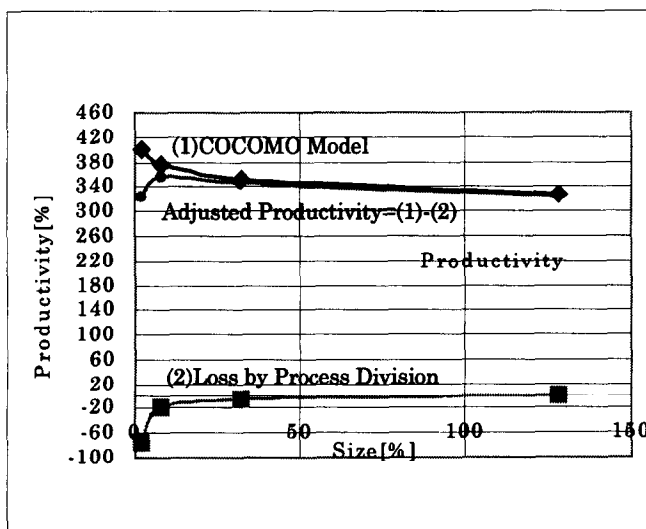


Fig. 4 Volume Economics of the ASP

the volume economics of the ASP as illustrated in Fig. 4. In this case, we assume $\delta=0.1$ based on our experience and the statistics in [Pitt93].

An important characteristic which Fig. 4 suggests is the scalability of productivity across a range of volume.

4 A MODEL OF AN AGILE SOFTWARE PROCESS

The ASP focuses on the following aspects of the software development process.

4.1 A Concurrent and Asynchronous Process

Development activities of large-scale software systems are inherently concurrent and asynchronous, that is, the progress of elaborating design is not uniform across the sub-systems or parts. As pointed out, the conventional linear synchronous process model does not fit well to this essential nature of software process. For example, Perry, Staudenmayer and Votta reported that developers spent 60% of their time in a development non-productively for a variety of reasons [Perr94]. We observed similar behaviors in our projects. As illustrated in Fig. 5, the ASP changes the underlying process model from a staged model to a concurrent asynchronous model.

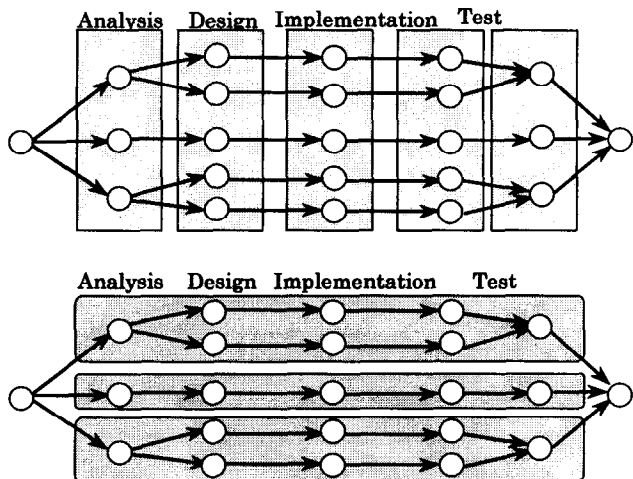


Fig. 5 A Concurrent Asynchronous Process

4.2 An Incremental and Iterative Process

Object-oriented development comes into real life. Among several software process models for the object-oriented development, we observed the following two common natures in the structure of the process models, as illustrated in Fig. 6 [Aoya94a, Booc95].

(1) Incremental macro process

From macro-scopic point of view, the software process is *incremental* or *spiral* in the sense that functions are divided into, say, sub-systems or a group of functions and *incrementally* developed in a series of releases.

(2) Iterative micro process

Within an increment, say, release or version, a micro life-cycle of the design and implementation is iterated until the sub-systems come to an acceptable level of functionality.

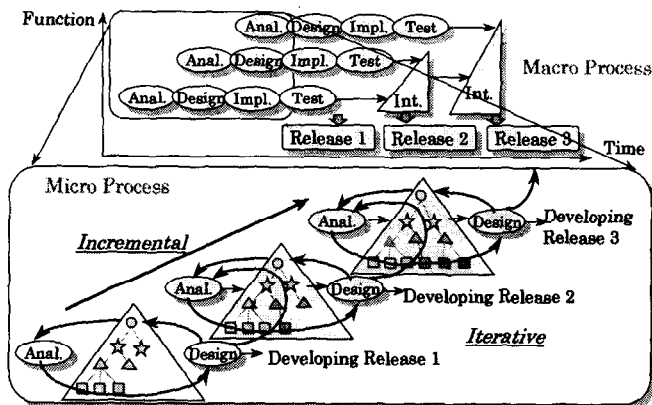


Fig. 6 An Incremental and Iterative Process

These characteristics are not unique to object-oriented development but are also observed in conventional development. However, due to the higher-modularity, object-oriented architecture makes it easy to introduce the incremental and iterative process model.

4.3 A Distributed Software Process

Software development organizations will be more and more distributed and global due to the globalization of business [Aoya94b]. Since the mid 80s, we have outsourced our development workforce to several development sites geographically distributed. This now accounts more than 60% [Aoya96a].

Distributed development brought about another significant impact on the software process and management, and challenged us to re-think the way we do software development. The ASP assumes global distributed development where even a single person in a remote place can participate. Therefore, the ASP does not imply any specific physical office configuration but is a *virtual software factory* over the Internet.

4.4 A People-Centered Process

The ASP is people-centered in the sense that people are not isolated elements in a large organization separated by stages but are contributors of organized into *small* teams. As illustrated in Fig. 7, each team is responsible for the complete work of a meaningful unit of the system, such as a functional enhancement, from analysis to test.

DeMarco and Lister pointed out people issues in software development [DeMa87]. Experience in hardware assembly processes also show that people are discouraged by long separated processes. In several assembly factories, reorganization of the linear processes into multiple independent small cellular processes improved employee satisfaction as well as productivity and quality. In the ASP, the concurrent and asynchronous process model enables people-centered process.

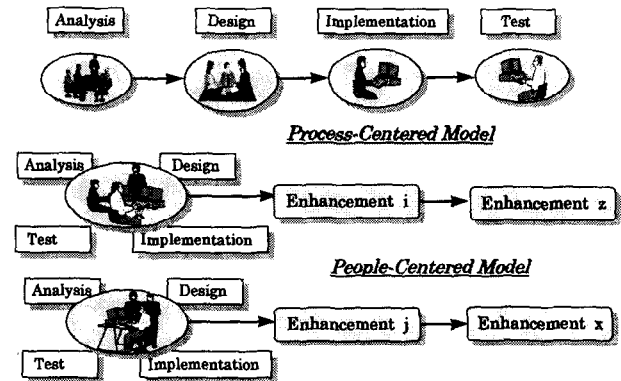


Fig. 7 People-Centered Process

5 AN AGILE SOFTWARE PROCESS SYSTEM

The ASP system is an integrated system for managing the ASP model as illustrated in Fig. 8. The software process model is particularly important in the ASP system since the structure and dynamic behavior of the ASP system are highly complex.

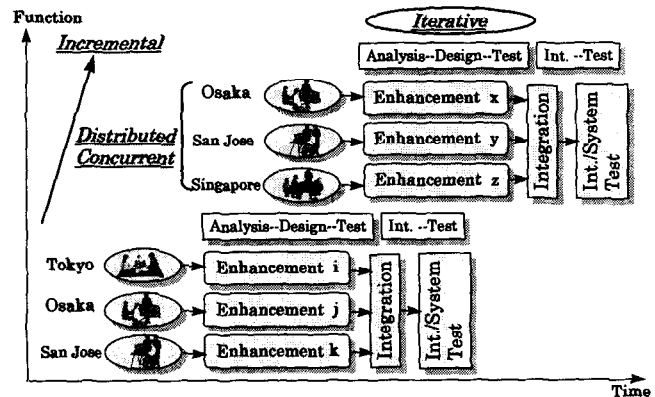


Fig. 8 A Process Model of the ASP System

5.1 Design Principles of the ASP System

To integrate a variety of software process techniques into workable ASP process, we have developed the following design principles.

(1) Public Private Processes: Encapsulation of Private Processes

The individual developer prefers his/her own control at the detailed level of process. A uniform enactment mechanism at such a detailed level may cause inflexibility and inefficiency. We set a management level which separates the levels of process abstraction into two categories: public process and private process. While public process is required to be visible to other developers, the execution of private process is encapsulated from other people. Each developer is allowed to change the enactment order of private processes in his/her own way. This concept is the same as *backlash* in mechanical engineering.

(2) Modular Processes

We have analyzed the entire life-cycle process except for

the details of individual developer's task with our visual process description language *YPL (YAC II-oriented Process description Language)* [Aoya90]. Based on the analysis, we divided the entire life-cycle process into 10 base processes. Furthermore, each base process is divided into some 10 sub-processes. Each sub-process is divided in such a way that one developer can complete it within a week for a nominal unit of workload.

(3) Concurrent and Cyclic Enaction with Fixed Cycle-Time

Multiple ASPs can be enacted concurrently and asynchronously in a release. A set of concurrent process is cyclically enacted over multiple releases with a fixed cycle-time. This enaction model makes it possible to develop multiple features with fixed workload because the fixed number of developers move from one enhancement to another. We set *six months of development cycle-time* that makes it possible to deliver each release every *three months*. Theoretically, the delivery cycle-time can be further shortened by curtailing the development cycle-time. However, shorter development cycle-time may sacrifice productivity and quality. Fixed cycle-time is the key to an ASP.

(4) Major and Minor Enhancements

Since some parts of the software are tightly coupled, it would not be productive to separate such parts into multiple enhancements. We introduced two time units of development namely *major enhancements* and *minor enhancements*. Six months and three months are respectively assigned to those upper streams of major enhancements and minor enhancements. These two different units of development time makes our process enaction more flexible.

(5) Alternative Release Cycles

Major enhancements usually produce a significant number of changes to be released. Therefore, two different units of enhancement resulted in two different units of release size, that is, *major release* and *minor release*. However, since the lower stream and release cycle are both fixed to three months, the major release usually brings about a heavy burden on integration and system test. Therefore, we allocate major releases and minor releases alternatively. This policy eases the stress on workload allocation and process management over multiple releases.

(6) Fixed Process Speed and Workloads

By fixed process cycle-times and incremental delivery, the ASP makes each development team maintain a fixed process speed. We believe that the change of process speed causes heavy stress on software developers.

5.2 A Design Process for the ASP System

Fig. 9 illustrates our two stage process for the design and implement of the ASP system. Process design deals with the tasks including the definition and improvement of software processes, and controlling the execution. Based

on process analysis, we have re-designed the software process in order to enable the ASP process and have developed a supporting process-centered software engineering environment.

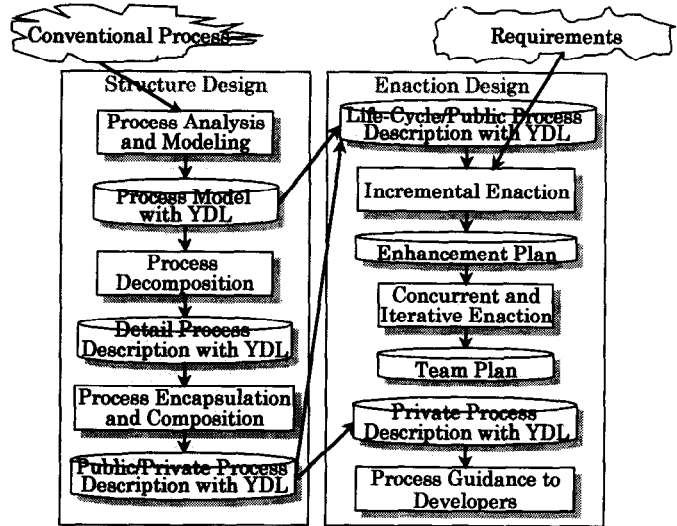


Fig. 9 A Two Stage Process for Process Design

6 AN ENVIRONMENT FOR THE ASP SYSTEM

An integrated software engineering environment is indispensable to implement and operate the ASP system. The environment has to provide a balanced support for process and product management. As suggested by the framework model in Fig. 10, the environment vertically integrates a wide variety of components at the following three layers.

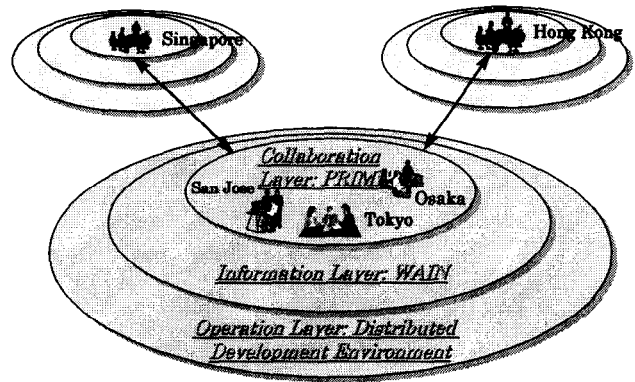


Fig. 10 A Framework of the ASP Environment

6.1 PRIME for the Collaboration Layer

This layer supports the collaborative work based on the process management techniques. As a base system we developed *PRIME (PRocess Information ManagEr)*, a process-centered software engineering environment. Fig. 11 illustrates three layers of distributed hierarchical architecture of PRIME [Aoya95].

PRIME provides the following functions.

- (1) To just-in-timely guide the individual developer software process defined in YPL on the windows of his/her workstation,
- (2) To support planning and executing software process,
- (3) To support collecting statistics at appropriate levels of process and organization through networks,
- (4) To visualize statistics from multiple view points at appropriate levels of process and organization, and
- (5) To control security.

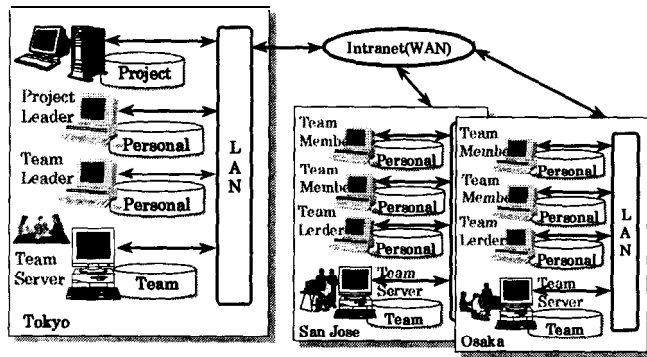


Fig. 11 The Architecture of PRIME

Fig. 12 shows a screen view of PRIME. The upper left sub-window shows a list of base processes. The upper right sub-window shows 'planning support. The progress of multiple teams and individual developers are respectively shown in the sub-window at left bottom and right bottom.

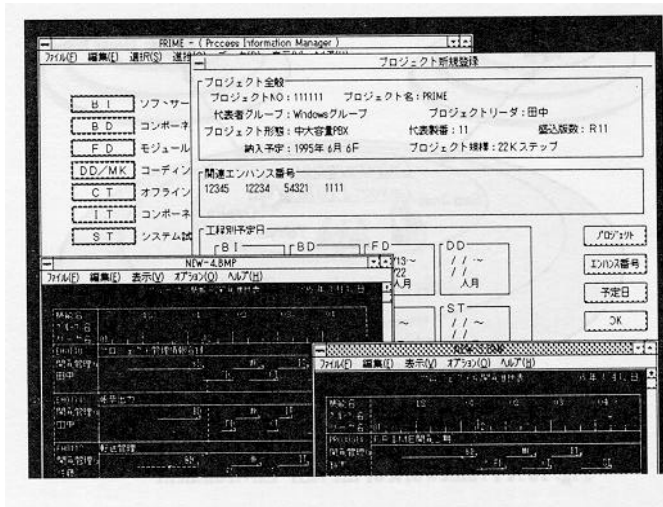


Fig. 12 A Screen View of PRIME

6.2 A WAIN for the Information Layer

An information layer is another leg of the tripod supporting the ASP system. However, conventional product management systems focus on either source code or well-structured design information such as data flow diagrams

and state transition diagrams [Aoya92]. Besides such information, we found a huge volume of design information that is not well-structured but semi-structured at best or *un-structured* at worst[Aoya96b]. The ASP system aims at managing not only well-structured information, but also semi/un-structured information, consistently along with software process. As a base system, we developed a WAIN (Wide Area Information Network) that manages structured and semi/un-structured design documents on the corporate intranet [Aoya96b]. Fig. 8 illustrates the architecture of the WAIN. Working with PRIME, WAIN provides the following functions.

- (1) To encapsulate the structure of various design documents and provide uniform access methods to the documents,
- (2) To support the creation, editing, retrieval and change control of design documents,
- (3) To enable developers to navigate across the distributed repositories, and
- (4) To control security.

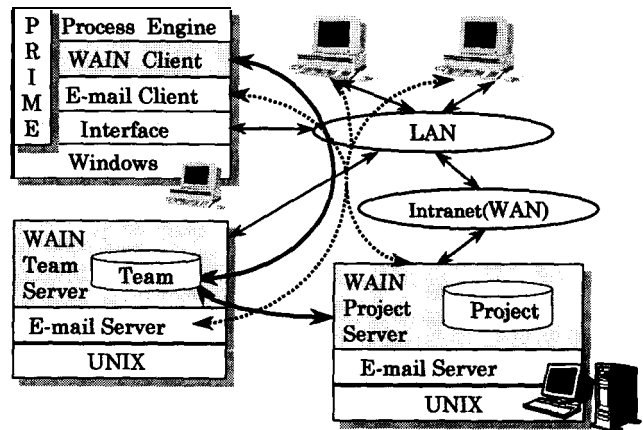


Fig. 13 The Architecture of WAIN

6.3 The Operation Layer

The operation layer is a infrastructure for the ASP system. We use UNIX servers and Windows clients connected together with the corporate intranet. Detailed discussions can be found in the literature [Aoya92, Aoya95].

7 EXPERIENCE AND LESSONS LERNED

7.1 Experience

The ASP process has been used in the development of a family of large-scale communication software systems. The system has evolved for ten years. As illustrated in Fig. 1, our process models have evolved too. At the stage documented below, more than 500 developers were involved the project.

While there are risks in implementing the ASP system, the benefits are significant and include:

- (1) Shortened development cycle-time by 75%.
- (2) Higher stability of work-loads,

- (3) Higher utilization of work-load, that is, developing large-scale, software systems with a fixed number of developers,
- (4) Higher flexibility to change of development plans, and
- (5) Higher quality by earlier feedback from the customers.

7.2 Evaluation

We evaluate our experience for last six years from both process and product points of view.

(1) Agile Release

As illustrated in Fig. 14, we have shortened the release cycle to three months which is a 75% reduction from a conventional release cycle of one year. This shorted release cycle brings about various benefits to us. For example, it avoids the use of patches for quick and dirty fixes of bugs and changes of functionality.

Table 1 shows the standard deviation within major releases and minor releases which suggest stable delivery of releases.

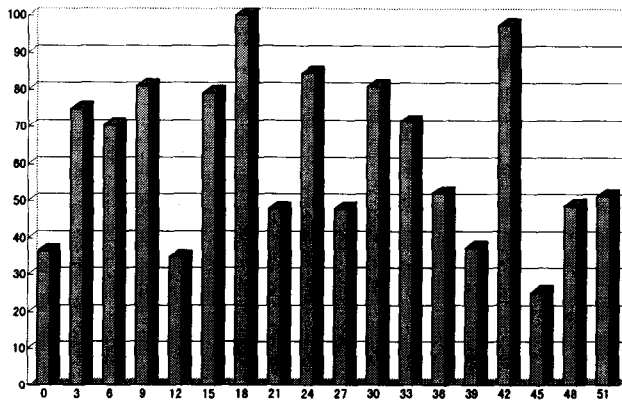


Fig. 14 Release Size

Table 1 Standard Deviations of Release Sizes and Workloads

Characteristics	Release Size		Workload
	Major Release	Minor Release	6 Month Cycle
Number of Samples	9	9	11
Standard Deviation	12.1[%]	20.5[%]	11.7[%]

(2) Linear Evolution

Unlike the conventional *non-linear* evolution model [Lehm85], the ASP made the objective software system have *linearly* evolved from a few hundreds KLOC to well over one million LOC as illustrated in Fig. 15. With the least square method, we estimated an evolution equation as:

$$y = 0.06x + 0.86 \quad (2)$$

Here, X and Y respectively means time in month and nominal size of software system.

This evolution is unique to the ASP system which can iteratively enhance the system with fixed size of code.

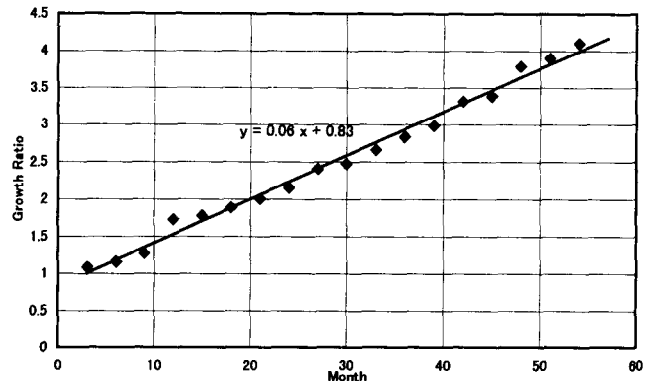


Fig. 15 Liner Evolutio of Software

(3) Stable Workloads

Fig. 16 illustrates two trend patterns of workloads. One is an example from the development of a previous software system named System 1 with conventional water-fall process. The other, Systems 2, is an example from the ASP. While the workload pattern of System 1 follows a Rayleigh curve [Putn80], the workload pattern of the ASP is flat.

Table 1 also shows the standard deviation of workload at every six months. Since the workload deviation 11.7% is smaller than the deviation of release size, it is interpreted that ASP process has been relatively well controlled. Change in workload, that is change of developers, causes productivity loss as well as stress to the project and corporate management. This is strong motivation to use the ASP which allows you to fix the number of developers for large-scale software development.

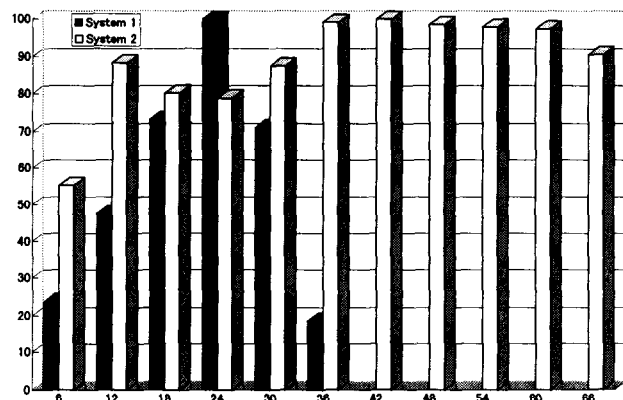


Fig. 16 Trends of Workloads

(4) Scalable Productivity

Figure 17 illustrates the relative productivity of each 18

releases against the size of the releases. It also shows a zone divided by two reference lines of the average plus and minus σ . As discussed in Section 2, the productivity of the ASP system is scalable across the size of releases. Figure 17 shows the scalability of productivity. It also suggests the modularity of the ASP process.

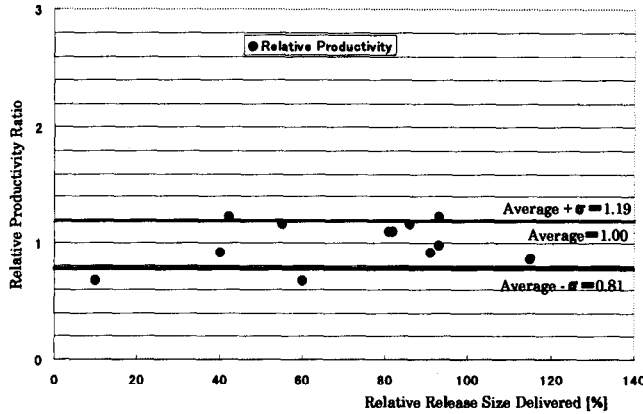


Fig. 17 Relative Productivity vs Release Size

(5) Higher Utilization Ratio of Work

The conventional water-fall process model follows a Rayleigh curve in the change of workload along with life-cycle. This change causes a loss of workload in the early and late stages of the life-cycle and usually requires work overtime in the peak at the middle of the life-cycle, which results in total productivity loss. The fixed workload of the ASP makes the utility of workload high. We define *URW* (Utility Ratio of Workload) to a trend pattern as:

$$URW = \frac{\text{Effective workload used for development}}{\text{Total workload}} \quad (3)$$

Since the trend pattern of the workload in the ASP system is fixed, the URW of the ASP system is theoretically 100%. However, the URW of the conventional water-fall process model is merely 44%. This means that the ASP system is possibly 56% leaner than the water-fall process model. Even taking some fluctuation of workload, 11.7% in our case, into consideration, the productivity of the ASP system can be higher than the water-fall model by 40%.

(6) Improving the Total Quality

We have evaluated the trend of the *TQR* (Total Quality Ratio) measured by:

$$TQR = \frac{\text{Total number of bugs injected in the release and identified after delivery}}{\text{Release size}} \quad (4)$$

The TQR has been improved by the factor of two for three years. This suggests that the ASP system can improve both quality and productivity by improving the software process.

7.3 Lessons Learned

(1) Ultimate Goal: From Cycle-Time Reduction to Process Improvement

Since the ASP process is much leaner than the conventional process models, it helps to expose various potential problems in our software process. Therefore, the ASP helped to improve our software process. We found that implementing the ASP process requires our underlying software process to be well-defined. We suggest not to apply the ASP process to an organization of the maturity level less than three. Therefore, we set up a taskforce with developers to analyze and improve our software process before implementing a series of process-based software engineering programs. This suggests that the ultimate goal of introducing the ASP process is improve the software process.

(2) Organizational Issue: PSET

We established a *PSET* (Project Software Engineering Team) inside the development organization. The missions of the PSET are broad, including process improvement, development of environment such as PRIME and WAIN, and technology transfer from research laboratory. Conventionally, staff organizations took these missions. However, it didn't work well since they lack the developer's point of view. We believe PSET is one of key success factors in the development and implementation of ASP system.

(3) Process Evolution

It took almost a decade to develop and implement the ASP system along with a series of process evolution starting with concurrent software process. Among various issues, the following two lessons should be noted;

- 1) Changing processes may take months to years,
- 2) Selecting an appropriate roadmap of process evolution is critical.

(4) People-Centered Approach to PSEE

Many PSEEs and groupware require rigorous management of process enactment especially on the order of process execution. However, we found such constraints cause inflexibility and loss of productivity. Therefore, we introduced an encapsulation policy and a backlash mechanism in the execution of software processes at the level of the individual developer. We believe our people-oriented philosophy in the design of PRIME is a key to designing PSEE.

(5) Light-Weight Processes

We applied the ASP system to small-scale software development projects of some 10 people. We found that the nature of the process was vastly different. They require a much leaner and simpler process. Therefore, we simplified the ASP process while retaining the basic process architecture.

8 RELATED WORKS

8.1 Comparison with Other Process Models

Table 2 summarizes the difference between the conventional water-fall process model and the ASP.

Table 2 Comparison of the Water-fall Process and the ASP

Characteristics	Water-fall Process	ASP
Principle	Volume	Time
Life-Cycle	Single Life-Cycle	Multiple Concurrent Life-Cycle
Release	Big Bang (Whole at a Once)	Incremental
Structure	Monolithic	Modular
Weight	Fat	Lean
Scalability	No: Depends on Product Size	Yes: Fixed with the Unit Size of Release
Workload	Raleigh Curve : Variable in Time	Fixed

Fig. 18 illustrates a classification of the major techniques of the ASP system, the Japanese software factory and the Toyota production system.

As illustrated, the ASP system is a brain child of the distributed concurrent process model and can be regarded as an advanced form of Japanese software factories [Aoya93, Aoya95].

It should be noted that the ASP helped to expose potential problems in the software process and motivated the improvement of the software process. This is similar to the experience in the implementation of the Toyota production system [Ohno78]. Although the initial goal of the ASP system is to shorten the development cycle-time, we found that the ultimate goal is to change the way we develop software systems.

8.2 Comparison with the Capability Maturity Model

The ASP and the CMM (Capability Maturity Model) [Hump89] share the same goal of improving the capability of software processes. However, their approaches are rather different. CMM provides a set of evaluation criteria and scenario for higher maturity, while the ASP system does a set of modularization and management techniques backed by an environment. For example, one of the key techniques of the ASP is time-based management. Therefore, the ASP emphasizes the stability of process execution that narrows the variety of completion time of processes. This leads to the same goal as CMM' higher maturity level: it narrows the variety of process performance [Paul95].

8.3 Comparison with the Synch-and-Stabilize Approach

The ASP and Microsoft's synch-and-stabilized approach [Cusu95] share some common principles such as incremental and concurrent process although the evolution

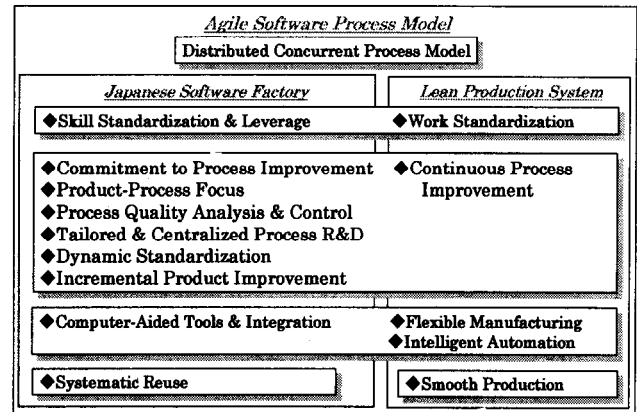


Fig. 18 ASP and Related Process Models

of those software processes appear different. The synch-and-stabilize approach evolved from a Hacker-style development and the ASP evolved from the Japanese software factories. However, these two approaches point to the common goals of shorter cycle-time and change of requirements, which could be needed in other development projects nowadays.

9 FUTURE STUDY

We are extending our research into three directions.

First, we are studying ways to transfer our technology to other projects.

Second, we are developing a set of technologies for the Software CALS (Continuous Acquisition and Life-cycle support) project which aims at integrating technologies for distributed collaborative development across corporations and countries over the Internet and intranets.

Finally, we are working on modeling and analyzing a component-based software development process and its economic model due to the rapid change of software architecture and widely spreading of componentware [Aoya97].

10 CONCLUSIONS

This article proposed a time-based software process model, the ASP (Agile Software Process), which was born from a decade of experience in an evolution of the software process for the development of a family of large-scale communication software systems. To implement the ASP is a challenge to us. It required us to re-think the way we did software development as well as we orchestrated various management techniques and environments for both software processes and products.

The ASP provided various benefits including shortening the development cycle-time by 75% from one year to three months. However, we realized that the ultimate goal of the ASP is to change the way we do software development. Unlike conventional process model based on the big-bang-delivery model and volume-base management, the ASP is based on an evolution model and on time-based management. It provides better predictability of product

evolution and process management. This leads to the conclusion that the ASP will bring about various new insights into software processes and environments, and change the way we develop software.

ACKNOWLEDGMENTS

The author would like to thank Mr. Kiyoo Nakamura and his colleagues of Fujitsu Limited for their support and cooperation. Thanks are due to Dewayne E. Perry for his support in polishing up the final manuscript.

REFERENCES

- [Agre86] W. W. Agresti, *New Paradigms for Software Development*, IEEE CS Press, 1986.
- [Aoya87] M. Aoyama, Concurrent Development of Software Systems: A New Development Paradigm, *ACM Software Eng. Notes*, Vol. 12, No. 3, Jul. 1987, pp. 20-24.
- [Aoya90] M. Aoyama, Distributed Concurrent Development of Software Systems: An Object-Oriented Process Model, *Proc. IEEE COMPSAC '90*, Nov. 1990, pp. 330-337.
- [Aoya92] M. Aoyama, et al., A Distributed Cooperative CASE Environment for Communications Software, *Proc. IEEE COMPSAC '92*, Sep. 1992, pp. 102-108.
- [Aoya93] M. Aoyama, Concurrent-Development Process Model, *IEEE Software*, Vol. 10, No. 4, pp. 46-55, Jul. 1993.
- [Aoya94a] M. Aoyama and S. Honiden, Making Object-Oriented Development Work, *J. IPSJ*, Vol. 35, No. 5, May 1994, pp. 451-460 (In Japanese).
- [Aoya94b] M. Aoyama, et al. (ed.), *The Rise of Software Industry and Technology in Asia-Pacific: Extra Edition of J. IPSJ*, Dec. 1994.
- [Aoya95] M. Aoyama, Management of Distributed Concurrent Development for Large-Scale Software Systems, *Proc. Asia-Pacific Software Engineering Conf. (APSEC) '95*, IEEE CS Press, Dec. 1995, pp. 158-167.
- [Aoya96a] M. Aoyama, Beyond Software Factories: Concurrent-Development Process and An Evolution of Software Process Technology in Japan, *J. of Information and Software Technology*, Vol. 38, No. 3, Mar. 1996, pp. 133-143.
- [Aoya96b] M. Aoyama, Sharing and Managing the Design Information in a Distributed Concurrent Development of Large-Scale Software Systems, *Proc. IEEE COMPSAC '96*, Aug. 1996, pp. 168-175.
- [Aoya97] M. Aoyama, Process and Economic Model of Component-Based Software Development, *Proc. 5th IEEE SAST (Symp. on Assessment of Software Tools)*, Jun. 1997, pp. 100-103.
- [Basi75] V. Basili and A. J. Turner, Iterative Enhancement: A Practical Technique for Software Development, *IEEE Trans. Software Eng.*, Dec. 1975, pp. 390-396.
- [Boeh81] B. W. Boehm, *Software Engineering Economics*, Prentice-Hall, 1981.
- [Boeh86] B. W. Boehm, A Spiral Model of Software Development and Enhancement, *ACM Software Eng. Notes*, Vol. 11, No. 4, Aug. 1986, pp. 22-42.
- [Booc95] G. Booch, *Object Solutions: Managing Object-Oriented Project*, Addison-Wesley, 1995.
- [Cusu91] M. A. Cusumano, *Japan's Software Factories*, Oxford University Press, 1991.
- [Cusu95] M. A. Cusumano and R. W. Selby, *Microsoft Secrets*, The Free Press, 1995.
- [DeMa87] T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*, Dorset House, 1987.
- [Fink94] A. Finkelstein, et al. (ed), *Software Process Modelling and Technology*, John Wiley and Sons, 1994.
- [Gold95] S. L. Goldman, et al., *Agile Competitors and Virtual Organizations: Strategies for Enriching the Customer*, Van Nostrand Reinhold, 1995.
- [Hump89] W. S. Humphrey, *Managing the Software Process*, Addison-Wesley, 1989.
- [Lehm85] M. M. Lehman and L. A. Belady, *Program Evolution*, Academic Press, 1985.
- [Naka92] K. Nakamura (ed.), *Ayumi: Continuous Improvement of Software Quality is Key at Fujitsu*, Japanese Union of Science and Eng., 1992 (In Japanese, English translation is available from the editor).
- [Ohno78] T. Ohno, *Toyota Production System*, Diamond Sha, 1978 (In Japanese, English translation is available from Productivity Press, 1988).
- [Oste87] L. J. Osterweil, Software Processes are Software Too, *Proc. 9th ICSE*, Mar. 1987, pp. 2-13.
- [Paul95] M. C. Paulk, et al., *The Capability Maturity Model: Guidelines for Improving the Software Process*, Addison Wesley, 1995.
- [Perr94] D. E. Perry, et al., People, Organizations, and Process Improvement, *IEEE Software*, Vol. 11, No. 4, Jul. 1994, pp. 36-45.
- [Pitt93] M. Pittman, Lessons Learned in Managing Object-Oriented Development, *IEEE Software*, Vol. 10, No. 1, Jan. 1993, pp. 43-53.
- [Putn80] L. H. Putnum (ed.) *Software Cost Estimation and Life-Cycle Control*, IEEE CS Press, 1980.